

Exercitiul 11 — Decorator: comportament adaugat prin impachetare

Cerința exercițiului 11 din modulul de design patterns.

Context

O aplicatie trimite notificari. Canalul de baza poate fi email sau SMS. Peste el vrei sa poti adauga, in orice combinatie:

- un **prefix** (`[URGENT]`),
- o **trunchiere** la o lungime maxima,
- o **jurnalizare** care scrie ce se trimite si pe ce canal.

Prima varianta la care se gandeste oricine e o clasa pentru fiecare combinatie: `SmsCuPrefix` , `SmsCuPrefixSiTrunchiere` , `EmailCuJurnal` ... Trei adaugiri inseamna opt clase. Patru inseamna saispzezece.

Decorator rezolva asta: fiecare adaugire devine o clasa care semneaza acelasi contract SI tine inauntru un obiect cu acelasi contract. Combinatiile se fac din obiecte, la rulare, nu din clase, la compilare.

Ai scris deja pattern-ul asta de trei ori — `ex1/LivrareCuReducere` , `ex6/ComisionCuPlafon` , `ex6/ComisionCuBonus` . Aici doar ii pui numele si il duci pana la capat.

Contractul

```
public interface INotificator
{
    string Nume { get; }
    void Trimite(string mesaj);
}
```

Ce ai deja scris

Clasa	Rol	Ce face
<code>NotificatorEmail</code>	ConcreteComponent	scrie <code>[EMAIL] mesaj</code>
<code>NotificatorSms</code>	ConcreteComponent	scrie <code>[SMS] mesaj</code> , dar arunca daca mesajul trece de 160 de caractere

Astea nu impacheteaza pe nimeni. Sunt capatul lantului.

Ce ai de implementat

Trei decoratori. Constructoarele si validările sunt scrise; tu completezi `Nume` si `Trimite`.

Clasa	Ce adauga
<code>CuPrefix</code>	lipeste prefixul in fata mesajului, apoi trimite mai departe
<code>CuTrunchiere</code>	daca mesajul e mai lung decat limita, il taie la limita, apoi trimite mai departe
<code>CuJurnal</code>	scrie <code>[JURNAL] catre <Nume interior>: <mesaj></code> inainte sa trimita mai departe

Regula pentru `Nume` : un decorator NU are voie sa-si intoarca un nume fix. Il compune din cel dinauntru:

```
public string Nume => interior.Nume + " + prefix";
```

Asa, `Nume` iti spune tot lantul dintr-o privire: `SMS + trunchiere + jurnal`.

Testul

In `Testare11` construiești și rulează:

1. `NotificatorEmail` simplu, cu un mesaj scurt.
2. `CuPrefix("[URGENT] ", NotificatorEmail)` — vezi prefixul in output.
3. Un mesaj de **peste 160 de caractere** trimis direct pe `NotificatorSms`, intr-un `try/catch` — vezi exceptia.
4. Acelasi mesaj lung, trimis prin `CuTrunchiere(160, NotificatorSms)` — de data asta trece.
5. **Cele doua ordini, una dupa alta**, cu acelasi mesaj lung:
 - `CuJurnal(CuTrunchiere(160, NotificatorSms))`
 - `CuTrunchiere(160, CuJurnal(NotificatorSms))`

Afiseaza inainte de fiecare `Nume` -le lantului.

Gata cand

- Punctul 5 da **doua rezultate diferite** si intelegi de ce: in primul caz jurnalul vede mesajul deja taiat, in al doilea il vede intreg. Un lant de decoratori nu e o multime, e o **ordine**.
- `Nume` afiseaza tot lantul, nu doar ultima adaugire.
- Nicaieri in cei trei decoratori nu apare `NotificatorEmail` sau `NotificatorSms` — lucreaza doar prin `INotificator`.
- Fiecare `Trimite` chiar deleaga: daca in vreunul nu apare `interior.Trimite(...)`, ai scris o strategie cu un camp nefolosit.

Constrangeri

- Fara `is`, `as`, `GetType()`, fara `switch / if` pe tip.
- Fara `List / Dictionary / LINQ`.
- Mesaje de eroare in engleza.
- `Testare11` chemat din `Program.cs`, ca la celelalte.

Bonus (de raspuns in scris, doua randuri)

1. Deschide `ex6/Models/ComisionCuBonus.cs`. `Nume` e hardcodat `"Cu bonus"`. Schimba-l sa compuna, ruleaza `Testare6` si spune ce se schimba in output. Fa la fel in `ComisionCuPlafon` si in `ex1/LivrareCuReducere`.
2. `ComisionCuPlafon` isi valideaza `comisionMax`, `ComisionCuBonus` nu-si valideaza `valFixa`. De ce conteaza, si care dintre cei trei decoratori de aici ar avea aceeasi problema daca n-as fi scris eu validările?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro